

HW2

Ethan Reinhart

November 2025

Review Questions

R6

Yes. An application can achieve reliable data transfer using UDP by implementing its own reliability mechanisms at the application layer.

In particular, the application can:

- Add sequence numbers to its messages.
- Have the receiver send acknowledgments (ACKs) back to the sender.
- Retransmit messages upon a timeout or upon receipt of a negative acknowledgment.
- Use checksums or stronger integrity checks to detect corruption.

UDP itself remains unreliable; the reliability is provided by the application-level protocol.

R14

- (a) Statement: Host A is sending Host B a large file over a TCP connection. Assume B has no data to send A. Host B will not send acknowledgments to Host A because it cannot piggyback ACKs on data.

Answer: False.

Even if Host B has no data to send, it must still acknowledge received segments. It sends pure ACK segments (with an ACK flag but no data payload). Piggybacking ACKs on data is an optimization, not a requirement for ACK generation.

- (b) Statement: The size of the TCP receive window (rwnd) never changes throughout the connection.

Answer: False.

The advertised receive window rwnd reflects the currently available space in the receiver's buffer. As the receiving application reads data out of the buffer, more space becomes available and rwnd increases; as data arrives faster than it is read, the buffer fills and rwnd decreases. Thus, rwnd is dynamic.

- (c) Statement: Suppose Host A is sending Host B a large file over a TCP connection. The number of unacknowledged bytes that A sends cannot exceed the size of the receive buffer.

Answer: True.

The effective sending window is

$$\text{SendWindow} = \min(\text{cwnd}, \text{rwnd}),$$

and rwnd is chosen so that the receiver's buffer does not overflow. Therefore, the total amount of in-flight (unacknowledged) data cannot exceed the advertised receive window, which is at most the receive buffer size.

- (d) Statement: Suppose Host A is sending a large file to Host B over a TCP connection. If the sequence number for a segment is m , then the sequence number for the subsequent segment is necessarily $m + 1$.

Answer: False.

TCP sequence numbers count bytes, not segments. If a segment carries L bytes of data and its sequence number is m , then the next segment's sequence number will be $m + L$, not necessarily $m + 1$ (unless $L = 1$).

- (f) Statement: Suppose the last measured SampleRTT in a TCP connection is 1 second. The current TimeoutInterval will necessarily be at least 1 second.

Answer: False.

TCP maintains a smoothed estimate:

$$\text{EstimatedRTT} \quad \text{and} \quad \text{DevRTT},$$

and sets

$$\text{TimeoutInterval} = \text{EstimatedRTT} + 4 \cdot \text{DevRTT}.$$

The most recent SampleRTT is only one measurement. If earlier samples were smaller, EstimatedRTT and DevRTT can both be less than 1 s, so the resulting TimeoutInterval can be less than the last SampleRTT .

- (g) Statement: Host A sends one TCP segment to Host B with sequence number 38 and 4 bytes of data. In this same segment, the acknowledgment number is necessarily 42.

Answer: False.

The sequence number field refers to data flowing in one direction (from A to B), while the acknowledgment number refers to data received in the opposite direction (from B to A). There is no requirement that

$$\text{ACK} = \text{SEQ} + \text{data length}$$

within the same segment. The ACK field indicates the next byte A expects from B, which is independent of what A is currently sending to B.

Problems

P22

Go-Back-N (GBN) with sender window size $N = 4$ and sequence numbers in $\{0, 1, \dots, 1023\}$. At time t , the next in-order packet expected by the receiver has sequence number k . The channel does not reorder messages.

- (a) Possible sets of sequence numbers in the sender's window.

Let b denote the base of the sender's window (oldest unacknowledged sequence number). Since the window size is 4, the sender window at any time is

$$\{b, b + 1, b + 2, b + 3\}.$$

Because the receiver currently expects k , it has successfully received all packets with sequence numbers $< k$ in order.

From the GBN behavior:

- The sender must have been able to send packet $k - 1$ at some point, so $k - 1$ cannot be greater than the highest sequence number in the sender's window:

$$k - 1 \leq b + 3 \quad \Rightarrow \quad b \geq k - 4.$$

- The base b cannot exceed k , because the sender cannot slide its window beyond k until it knows that all packets up to $k - 1$ have been acknowledged.

Thus

$$k - 4 \leq b \leq k.$$

Consequently, the possible sender windows are:

$$\begin{aligned} &\{k - 4, k - 3, k - 2, k - 1\}, \\ &\{k - 3, k - 2, k - 1, k\}, \\ &\{k - 2, k - 1, k, k + 1\}, \\ &\{k - 1, k, k + 1, k + 2\}, \\ &\{k, k + 1, k + 2, k + 3\}. \end{aligned}$$

- (b) Possible acknowledgment numbers currently propagating from receiver to sender.

The receiver is expecting k , so:

- The largest cumulative ACK it can generate is $k - 1$ (meaning all bytes or packets up to and including $k - 1$ have been correctly received).
- Earlier ACKs (for smaller highest in-order sequence numbers) may still be in the channel if they were delayed.

Given the window size is 4, the receiver's last four in-order packets correspond to sequence numbers $k - 4, k - 3, k - 2, k - 1$. Their ACKs may still be in flight. Thus the possible ACK values propagating back from receiver to sender are:

$$\{k - 4, k - 3, k - 2, k - 1\}.$$

P40

We consider the canonical TCP Reno congestion window evolution plot (Figure 3.61) showing congestion window size vs transmission round.

- (a) Transmission rounds in which TCP is in slow start.

In slow start, the congestion window (cwnd) increases approximately exponentially (doubling each RTT). From the figure:

- Rounds 1 through 6 show exponential growth up to the initial threshold.
- After a timeout around round 22, cwnd is reset to 1 and again grows exponentially from rounds 23 through 26.

So TCP is in slow start during rounds

1–6 and 23–26.

- (b) Transmission rounds in which TCP is in congestion avoidance.

In congestion avoidance, cwnd increases approximately linearly (by about one MSS per RTT). From the figure:

- After slow start ends around round 6, cwnd grows linearly from rounds 6 to 16.
- After the fast recovery from the triple-duplicate ACK loss event, cwnd again grows linearly from rounds 17 to 22.

Hence TCP is in congestion avoidance during rounds

6–16 and 17–22.

- (c) Loss detection after the 16th round: triple duplicate ACK or timeout?

Immediately after the 16th round, the congestion window is reduced from about 42 segments to roughly half that value (around 21), not to 1 segment.

Under TCP Reno:

- A timeout would reset cwnd to 1.
- A triple duplicate ACK reduces cwnd to approximately half its previous value (and enters fast recovery or congestion avoidance).

Therefore, the loss after the 16th round was detected by triple duplicate ACKs, not by a timeout.

- (d) Loss detection after the 22nd round: triple duplicate ACK or timeout?

After the 22nd round, the congestion window in the figure drops all the way down to 1 MSS. This behavior corresponds to a timeout event in TCP Reno. Triple duplicate ACKs would have reduced cwnd roughly by half, not to 1.

- (e) Initial value of ssthresh at the first round.

The slow start threshold ssthresh is the cwnd value at which TCP switches from slow start (exponential growth) to congestion avoidance (linear growth). In the figure, cwnd grows exponentially until it reaches 32 segments, then the growth becomes linear. Thus the initial ssthresh is

$$\text{ssthresh} = 32.$$

- (f) Value of ssthresh at the 18th round.

The first loss event (detected via triple duplicate ACKs) occurs around the 16th round, when cwnd is approximately 42 segments. Under TCP Reno, on triple duplicate ACK:

$$\text{ssthresh} \leftarrow \frac{\text{cwnd}}{2}.$$

Thus

$$\text{ssthresh} = \frac{42}{2} \approx 21.$$

So at the 18th round, ssthresh is 21.

- (h) During which transmission round is the 70th segment sent?

We count the total number of segments sent in each round, using the cwnd sizes (in segments) from the initial slow start region:

Round	cwnd (segments)	Cumulative segments sent
1	1	1
2	2	1 + 2 = 3
3	4	3 + 4 = 7
4	8	7 + 8 = 15
5	16	15 + 16 = 31
6	32	31 + 32 = 63
7	≈ 33	63 + 33 = 96

By the end of round 6, 63 segments have been sent (segments 1–63). In round 7, we send segments 64 through 96, so the 70th segment is sent during round 7.

- (i) After the 26th round, suppose TCP Reno detects a loss via triple duplicate ACKs. What are cwnd and ssthresh just after?

From the figure, near the 26th round, cwnd is about 8 segments. For TCP Reno, on triple duplicate ACK:

$$\text{ssthresh} \leftarrow \frac{\text{cwnd}}{2} \Rightarrow \text{ssthresh} = \frac{8}{2} = 4.$$

During fast recovery, Reno sets:

$$\text{cwnd} \leftarrow \text{ssthresh} + 3 = 4 + 3 = 7.$$

So, immediately after detecting the loss via triple duplicate ACKs:

$$\text{ssthresh} = 4, \quad \text{cwnd} = 7.$$

- (j) Now suppose TCP Tahoe is used (instead of Reno), and triple duplicate ACKs occur at the 16th round. What are ssthresh and cwnd at the 19th round?

At the time of loss (round 16), cwnd is about 42 segments. Tahoe differs from Reno in that on triple duplicate ACK it always drops back to slow start:

$$\text{ssthresh} \leftarrow \frac{\text{cwnd}}{2} = 21, \quad \text{cwnd} \leftarrow 1.$$

Then Tahoe uses slow start from cwnd = 1:

Round 17 : cwnd = 1,

Round 18 : cwnd = 2,

Round 19 : cwnd = 4.

Thus, at the 19th round:

$$\text{ssthresh} = 21, \quad \text{cwnd} = 4.$$

- (k) Again assuming TCP Tahoe and a timeout at the 22nd round, how many packets are sent from rounds 17 through 22 inclusive?

From the behavior described above (Tahoe starting from cwnd = 1 at round 17 and using slow start until the timeout at round 22):

Round 17 : cwnd = 1 $\Rightarrow$ 1 segment,

Round 18 : cwnd = 2 $\Rightarrow$ 2 segments,

Round 19 : cwnd = 4 $\Rightarrow$ 4 segments,

Round 20 : cwnd = 8 $\Rightarrow$ 8 segments,

Round 21 : cwnd = 16 $\Rightarrow$ 16 segments,

Round 22 : cwnd = 21 $\Rightarrow$ 21 segments.

Total number of packets sent from rounds 17 through 22 is:

$$1 + 2 + 4 + 8 + 16 + 21 = 52.$$

So there are 52 packets in total.